

A Scalable and Robust Compilation Framework for Emitter-Photonic Graph State

Xiangyu Ren^{†*}, Yuexun Huang[‡], Zhiding Liang^{§*}, Antonio Barbalace[†]

[†] University of Edinburgh, Edinburgh, Scotland, UK

[‡] University of Chicago, Chicago, IL, USA

[§] Rensselaer Polytechnic Institute, Troy, NY, USA

Abstract—Quantum graph states are critical resources for various quantum algorithms, and also determine essential interconnections in distributed quantum computing. There are two schemes for generating graph states — *probabilistic scheme* and *deterministic scheme*. While the all-photonic *probabilistic scheme* has garnered significant attention, the emitter-photonic *deterministic scheme* has been proved to be more scalable and feasible across several hardware platforms.

This paper studies the GraphState-to-Circuit compilation problem in the context of the *deterministic scheme*. Previous research has primarily focused on optimizing individual circuit parameters, often neglecting the characteristics of quantum hardware, which results in impractical implementations. Additionally, existing algorithms lack scalability for larger graph sizes. To bridge these gaps, we propose a novel compilation framework that partitions the target graph state into subgraphs, compiles them individually, and subsequently combines and schedules the circuits to maximize emitter resource utilization. Furthermore, we incorporate *local complementation* to transform graph states and minimize entanglement overhead. Evaluation of our framework on various graph types demonstrates significant reductions in CNOT gates and circuit duration, up to 52% and 56%. Moreover, it enhances the suppression of photon loss, achieving improvements of up to $\times 1.9$.

I. INTRODUCTION

Quantum information processing has the potential to revolutionize computation, communication, cryptography, sensing, and imaging. Within quantum information, entanglement plays a vital role as it serves as the foundation for interconnections between qubits. *Graph state* – a highly entangled quantum state, is the critical resource for quantum error correction (QEC) [1], measurement-based quantum computing (MBQC) [2], and various other applications. Moreover, it enables distributed quantum computing [3], [4] and quantum networking [5], acting as a paradigm for non-local qubit interactions. For instance, the *Bell state* is a special case of a graph state.

Before graph states can be utilized for various applications, they must first be generated on quantum computers or quantum network nodes. Graph state generation has predominantly focused on the *probabilistic scheme* [6], which relies on a sequence of *fusion gates* and photon interference. This all-photonic approach is widely studied for its relevance in optical quantum computing. Several previous compilation studies [7], [8], [9] for photonic MBQC are rooted in the *probabilistic scheme*. However, such approach suffers from an exponential demand for resources, severely limiting the scalability of the target graph states. Additionally, the randomness and uncertainty associated with fusion gates significantly increase the complexity of the compilation algorithms.

An alternative hardware solution is the *deterministic scheme*, where *photon qubits* are directly generated by emitters [10]. In this scheme, a set of interacting *emitter qubits* is employed to induce entanglement among photon qubits as they are emitted. Owing to better scalability and advancements in engineering, the *deterministic*

scheme is attracting increasing attention. Emitter-based graph states have been demonstrated across various hardware platforms, including silicon quantum dots [11], [12], nitrogen vacancy (NV) color centers [13], [14], and Rydberg atoms [15], achieving scales of up to 14 qubits [16].

The generation process of an emitter-based graph state could be formalized by a quantum circuit model. Constructing the quantum circuit for the generation of a specific graph state can be referred to as the **GraphState-to-Circuit compilation** — which is the problem we focus on in this paper. Such a compilation task has distinct constraints compared to previous quantum compilers designed for superconducting [17], [18], [19], [20], [21], [22], trapped-ion [23], all-photonic [7], Bosonic quantum [24], or neutral atom [25] platforms. Shown in Figure 1.a, the circuit model for graph state generation [26], [27] contains two types of qubits: *emitter qubit* and *photon qubit*. Entanglement is created via emitter-emitter interactions (e.g., emitter-emitter CNOTs), and the entanglement connections are transduced to photons through the photon emission process. However, for the purpose of being *deterministic*, photon-photon interactions are not allowed, which significantly differentiates this approach from the all-photonic *probabilistic scheme*. Moreover, the gates for different types of qubits are implemented based on completely different physical operations, leading to significantly varying gate durations. These particular constraints necessitate a specific compiler.

A series of research efforts attempt to address the compilation problem for emitter-based graph state generation. Li et al. [26] propose a method to minimize the emitter resources required for graph state generation. However, their approach does not consider circuit depth and suffers from photon loss issues due to the prolonged hardware operations of emitter CNOTs. Kaur et al. [28] explore loss-aware generation for repeater graph states (RGS), but their method does not fully exploit resource utilization for arbitrary graph states. Ghanbari et al. [27] leverage *local complementation* to optimize graph states and reduce generation overhead. Nevertheless, finding the optimal *local complementation* is #P-complete [29], which limits the scalability of this optimization. Furthermore, none of these works provide a feasible compilation scheme for large-scale graphs. Hindered by high complexity and an exponential solution space, optimal solutions for random graph states can only be achieved at the scale of a few qubits.

To address the above limitations, we propose a novel compilation framework for the graph state generation task, adopting a divide-and-conquer strategy to tackle the scalability problem. First, we partition the graph state into subgraphs, enabling optimal compilation for each subgraph with low overhead, while integrating *local complementation* for optimization. Next, we compile the subgraphs based on a comprehensive objective function that accounts for the characteristics of actual quantum computing hardware. Finally, during subgraph recombination, we design a circuit scheduling scheme to exploit emitter reuse among subgraphs, thereby maximizing qubit

* Corresponding to: Zhiding Liang (liangz9@rpi.edu) and Xiangyu Ren (xiangyu.ren@ed.ac.uk).

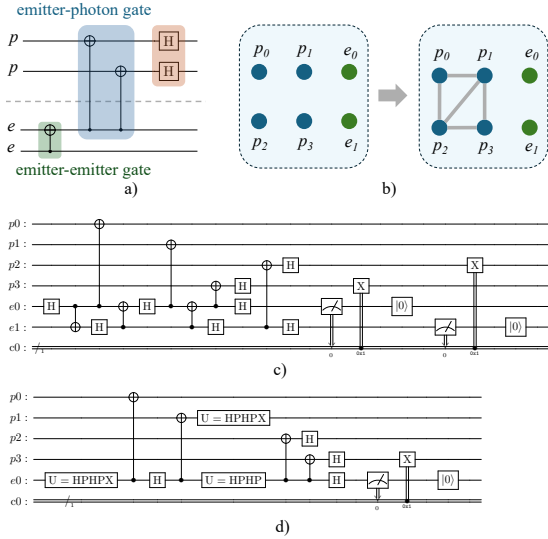


Fig. 1: The background on GraphState-to-Circuit Compilation. a) The constraints of a *deterministic* graph state generation, with p as photon and e as emitter. For deterministic purpose, only emitter-emitter CNOT in green and emitter-photon CNOTs (emissions) in blue are allowed. b) Generation of a graph state on photon qubits $p_0 - p_3$. c) The circuit for generating the graph state in b), as a result of compilation. d) An optimized circuit for the same graph state, with fewer emitters utilization and fewer CNOTs engaged compared to c).

resource utilization and reducing photon loss accumulation during generation. Our framework is evaluated on various types of graph states and compared with the best-known method [30]. It outperforms the baseline, achieving on average a 30% (up to 52%) reduction in emitter-emitter CNOTs, a 38% (up to 56%) reduction in circuit duration, and a suppression of photon loss by $\times 1.4$ (up to $\times 1.9$).

Our key contributions are as follows:

- 1) We are the first to study emitter-based graph state generation from a quantum compiler perspective. We formulate the compilation constraints, perform a comprehensive analysis of previous methods, and identify the objectives to consider based on realistic hardware implementations.
- 2) We propose a scalable compilation method by partitioning the target graph state into subgraphs and compiling these subgraphs. This method incorporates *local complementation* to reduce the critical overhead – emitter-emitter CNOT.
- 3) We design a scheduling scheme that maximizes the utilization of emitter qubit resources. It minimizes circuit duration, thereby reducing the accumulation of photon loss.

II. BACKGROUND

A. Quantum Graph State

A quantum graph state $|G\rangle$ is a specific type of multipartite entanglement state that is intrinsically associated with a graph $G = (V, E)$, where V is the set of vertices and E is the set of edges connecting these vertices. In graph G , a vertex v represents a physical qubit, whereas an edge e indicates the entanglement between the two qubits. The formal definition of a graph state $|G\rangle$ is given by:

$$|G\rangle = \prod_{(i,j) \in E} CZ_{(i,j)} |+\rangle^{\otimes |V|} \quad (1)$$

Here, $|+\rangle^{\otimes |V|}$ denotes the tensor product of $|V|$ copies of $|+\rangle$ state at all the vertices of G , and $CZ_{(i,j)}$ is the controlled-Z gate operation applied to the pair of qubits corresponding to the edge $(i, j) \in E$.

Typically, the generation of a graph state is achieved through two steps: 1) Prepare a set of qubits in initial states, corresponding to all the vertices in V ; 2) For each pair of qubits where the entanglement is desired, apply a CNOT/CZ interaction. Note that for a graph state, all CNOT/CZ gates are commutable, allowing them to be arranged in arbitrary sequences or applied in parallel, providing abundant flexibility for circuit scheduling. Here we provide an example of graph state, shown on the right side of Figure 1.b), representing the entanglement among photon qubits p_0, p_1, p_2, p_3 .

B. Emitter-Based Graph State Generation

Problem Formulation. Here we provide a description of the circuit model for emitter-based graph state generation, including all elementary operations required for the process, as illustrated in Figure 1.a-c, and an example of optimization in Figure 1.d:

- 1) Two types of qubits are involved: *emitter qubit* and *photon qubit*. Emitter qubits are initialized to state $|0\rangle$, while photon qubits do not exist physically before emission and are represented as $|0\rangle$.
- 2) Entanglement is induced by emitter-emitter CNOT, and the entanglement connection are transduced to emitted photon via emission process.
- 3) The emission of a photon is modeled by an emitter-photon CNOT between the emitter and emitted photon, being always the first gate applied on each photon qubit.
- 4) For *deterministic* purpose, the photon-photon interactions are not allowed. Only single-qubit gates and measurement can be applied on emitted photon qubits.
- 5) The final result should be a graph state entangled across n photons (n = size of the graph), while all emitters return to $|0\rangle$ state upon completion.

Hardware Characteristic. The emission and entanglement operations are implemented differently across hardware platforms. Here we use the quantum dots platform [10], [31] as an example. The emitter-emitter CNOT is realized through electron exchange coupling between quantum dots, with a gate time of 1-10 ns and a fidelity ≥ 0.99 . In contrast, the emitter-photon CNOT (emission) has a significantly shorter duration of ~ 0.1 ns. Given this distinction in gate time, circuit scheduling for emitter-based graph state generation becomes a non-trivial problem.

C. Time-Reversed Model for Graph State Generation

Our compilation algorithm is built upon a **time-reversed model** introduced in previous work [26]. In this time-reversed context, photon emission is replaced with a hypothetical "photon absorption", while emitter-emitter interaction correspond to "disentangling" between emitters. Using these reversed operations, we can derive

Description	Time-reversed operation	Time-forward circuit
Swapping a free emitter and a photon		
An emitter absorb a dangling photon		
A dangling emitter absorb a photon		
An emitter absorb a photon with same neighbors		
Remove connection between two emitters		

Fig. 2: Operations of the time reversed graph reduction model.

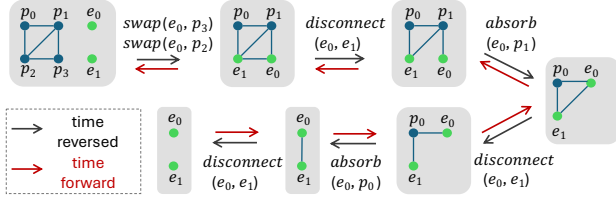


Fig. 3: An example of the time-reversed reduction sequence (black arrows), corresponding to the circuit in Figure 1.c (red arrows).

a virtual sequence that reduces a target graph state to initial state. Hence, reversing this sequence yields the required circuit to generate target state from initial state. We specify the following reversed operations [28], which are also illustrated in Figure 2:

- a) *Swapping with free emitter*: An arbitrary photon in the graph state can be replaced by a free emitter in $|0\rangle$ state.
- b) *Photon absorption*: If an emitter is entangled with a photon in the graph, it can absorb the photon under certain conditions. The specific conditions for photon absorption are illustrated in Figure 2.
- c) *Emitter disentangle*: Remove the entanglement connection (represented as an edge in the graph) between two emitters.

Based on these operations, our compiler determines the optimal time-reversed reduction sequence of target graph state (e.g. Figure 3), then reverses this sequence to generate the desired circuit. We recommend Kaur et al. [28] for a more comprehensive background.

D. Optimizing Graph State via Local Complementation

By applying local Clifford (LC) gates to a graph state, it can be transformed into a set of alternative graph states that are LC-equivalent with original graph state. Hence, generation of the target graph state can be redefined as generating an LC-equivalent graph state. If the LC-equivalent graph has fewer edges, this transformation can reduce the entanglement overhead, with the only cost being the addition of single-qubit gates to the circuit.

Finding LC-equivalent graph state can be formalized as applying *local complementation* to the graph [27], [29]. Specifically, local complementation of a vertex v in the graph modifies the edges within the neighborhood on v . In the neighborhood of v , edges that are present will be removed, and edges that are missing will be added (Figure 4). A proper local complementation of target graph state can significantly reduce the entanglement overhead. However, determining the optimal local complementation has been proven to be #P-complete [32], making it infeasible to solve in polynomial time.

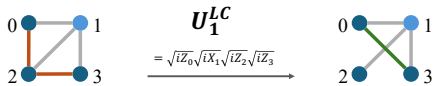


Fig. 4: Leveraging local complementation to optimize the graph state. In this example, a Clifford unitary is applied on photon 1 (U_1^{LC}), leading to a local complementation of the corresponding vertex 1, acting on neighborhood vertices 0, 2, 3.

III. MOTIVATION

A. Challenges and Solution

Given the constraints and optimization opportunities, GraphState-to-Circuit compilation is a complex and non-trivial task. In this section, we discuss the associated challenges and propose our solution.

Challenge 1: There is an exponential overhead in searching the optimal circuit of graph state generation. Additionally, finding the best local complementation for target graph state is a #P-complete problem. As a result, the compilation is **not scalable**.

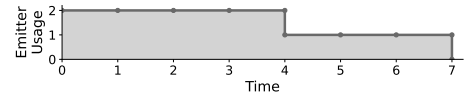


Fig. 5: The emitter usage number over time (timescale simplified), for a graph state generation circuit.

As we mentioned in Section II.A, the commutation of CNOT/CZ in graph state generation allows photon emission in an arbitrary sequence. As a result, the compilation overhead grows exponentially when exhaustively searching all possible circuits. For the state-of-the-art compiler GraphiQ [30], the runtime exceeds 10^3 seconds when solving a linear cluster state with more than 10 qubits. Furthermore, when realistic optimization targets such as circuit duration and loss rate are considered instead of simply minimizing the #CNOT, the runtime becomes even worse.

Solution: We partition the target graph state into several subgraphs, each with a feasible size for finding the optimal generation circuit. These subgraphs are treated as *leaves* and are compiled separately with designated optimization. Meanwhile, those inter-subgraph entanglements are treated as *stems*, compiled into an independent circuit and integrated into the final result.

Challenge 2: In hardware-aware scenarios, there are **various factors affecting the quality** of compiled circuits.

Based on the hardware characteristic of quantum dots emitter in Section II.B, several optimization targets must be considered: 1) A limited number of emitters, leading to prolonged circuit execution; 2) Accumulation of photon loss over time, which may render photons unusable; 3) Imperfect emitter-emitter CNOTs, affecting fidelity of the target state. Previous research has only considered one of these factors [26], resulting in impractical outcomes.

Solution: We construct a cost function that combines multiple optimization objectives. Specifically, we extract the gates duration based on hardware settings, and use them to calculate the photon loss duration of generation circuit as the cost function.

Challenge 3: When an emitter is entangled in the graph state, it may remain idle due to the lack of available operations. This results in **insufficient utilization of emitters**, thereby increasing the circuit duration.

Figure 5 provides an example of the emitter usage curve of a circuit, illustrating the number of utilized emitters during the generation process. It can be observed that, for certain parts of the circuit, the utilization of emitters does not reach the maximum capacity. In other words, the emitter resources are not fully utilized. This under-utilization becomes more pronounced as the scale of the target graph state increases, resulting in a generation circuit with significantly longer duration.

Solution: We schedule the subgraphs circuits by arranging them on the timeline. Based on the emitter utilization curve of each subgraph, we can share emitter qubits among different subgraphs, ensuring maximum utilization of emitters at every time slot. Briefly, we exploit the circuit-level parallelism and emitter reuse.

B. Insight: Divide-and-Conquer

The framework we propose adopts a *divide-and-conquer* strategy, inspired by the natural *stem-leaf* structure of a tree graph, but also applies to arbitrary graph structure. A straightforward illustration of this *stem-leaf* structure can be found in the left part of Figure 6. By partitioning the target graph state into several subgraphs [33],

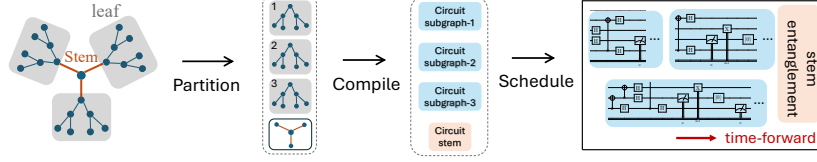


Fig. 6: The overview of our framework.

we treat the entanglements within each *leaf* as sub-problems, and finally recombine them through the compilation of inter-subgraph entanglements as the *stem*. The overview of framework is in Figure 6.

Our insight within this strategy is that finding the global optimal solution of graph state generation circuit is highly complex. However, optimizing each sub-problem individually can reduce overhead and achieve a local-minimum. Another challenge is determining an appropriate partitioning scheme for any graph state, which will be discussed in Section IV.A. Additionally, our circuit scheduling approach reduces redundancy by treating each subgraph as a coarse-grained basic unit.

IV. FRAMEWORK DESIGN & IMPLEMENTATION

A. Graph State Partitioning

We present the design of the first part in our compilation framework, where the target graph state is partitioned into subgraphs. To find an appropriate partitioning scheme, the following considerations should be addressed:

Reduce interconnections among subgraphs. The target graph state is partitioned into subgraphs, and each subgraph is compiled with lower runtime overhead. After obtaining the optimal generation circuit for each subgraph, they need to be connected via inter-subgraph entanglements (edges). In the resulting circuit that combines all subgraphs, these inter-subgraph entanglements will be compiled into emitter-emitter CNOTs, which are expensive operations on the hardware. Thus, the objective of graph state partitioning is to minimize the number of interconnection edges among different subgraphs.

Optimize graph partitioning with local complementation. Taking the graph state in Figure 7 as an example, normally at least 5 interconnecting edges must be cut to divide it into two subgraphs, shown in (a). However, by applying local complementation on vertices α and β , only 1 interconnecting edge remains for the partitioning, shown in (b). Thus, finding the appropriate local complementation is crucial not only for reducing the overall number of edges, but also for minimizing the partitioning cost. Due to the exponential search space for local complementations, we limit the number of LC operations to l , resulting in a depth-limited search. This restriction not only simplifies the search process, but also limits the additional circuit depth introduced by the local complementation operations.

Graph state partition searcher with depth-limited LC. We leverage a mix integer programming (MIP) solver to simultaneously find the best local complementation and graph partitioning solution. Our MIP model is described as follows:

a) *Constant Parameters:* Given the target graph state $G = (V, E)$, the vertices are represented as $v \in V$ and the edges as $e \in E$. Assuming the number of local complementation (LC) operations is limited to l , we define a sequence $1, 2, \dots, l$ to mark the status of the graph state at each step. The subgraphs partitioned from G

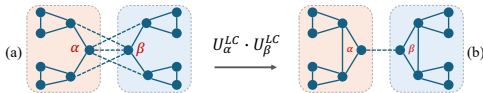


Fig. 7: LC optimization for graph state partitioning.

are represented as the set $\{g\}$, with size limitation $|g| \leq g_{max}$. Additionally, the total number of subgraphs is given by $\lceil |G|/g_{max} \rceil$.

b) *Variables:* First, we define our edge status in each step t by $e_{v_1, v_2, t}$ ($t \in 1, \dots, l$), and vertices in each subcircuit by $y_{v, g}$:

$$e_{v_1, v_2, t} = \begin{cases} 1, & \text{if vertex } v_1, v_2 \text{ are adjacent at step } t \\ 0, & \text{otherwise} \end{cases}$$

$$y_{v, g} = \begin{cases} 1, & \text{if vertex } v \text{ is partitioned in subgraph } g \\ 0, & \text{otherwise} \end{cases}$$

Then, we set the local complementation in each step as $x_{v, t}$.

$$x_{v, t} = \begin{cases} 1, & \text{if local complementation applied on } v \text{ at step } t \\ 0, & \text{otherwise} \end{cases}$$

c) *Constraints:* For local complementation, we only apply it on single vertex at each step (or skip this step), thus we have:

$$\sum_{v \in V} x_{v, t} \leq 1, \quad \forall t \in \{1, \dots, l\} \quad (2)$$

Most importantly, the local complementation at step t will add or remove the edges, resulting in a change at step $t + 1$. Suppose the local complementation is applied to vertex v_0 ($x_{v_0, t} = 1$), and v_1, v_2 being two of its adjacent vertices ($e_{v_0, v_1, t} = e_{v_0, v_2, t} = 1$). Then, there should be a change to the edge e_{v_1, v_2} from step t to $t + 1$:

$$x_{v_0, t} * e_{v_0, v_1, t} * e_{v_0, v_2, t} \leq |e_{v_1, v_2, t} - e_{v_1, v_2, t+1}| \quad (3)$$

In addition, the size of each subgraph is restricted by:

$$\sum_{v \in V} y_{v, g} \leq g_{max}, \quad \forall g \in \{g\} \quad (4)$$

d) *Objective Function:* Since we hope to minimize the interconnecting edges among subgraphs, our objective function is the number of such edges, represented by K . We use $\sum_{v \in \{g\}} y_{v, g} * y_{v_2, g}$ to determine if v_1, v_2 are in the same subgraph, then we have:

$$K = \sum_{\forall v_1, v_2 \in V} e_{v_1, v_2, l} * (1 - \sum_{v \in \{g\}} y_{v_1, g} * y_{v_2, g}) \quad (5)$$

Overall, the MIP model can be formulated as

$$\begin{aligned} & \text{minimize objective } K (\text{Eq. 5}) \\ & \text{s.t. constraint Eqs. (2, 3, 4)} \end{aligned}$$

B. Optimizing Circuit Depth for Subgraph Generation

Optimizing Targets. In this section, we introduce our compilation method for the subgraphs. Unlike previous works that focus on optimizing either the minimal emitter resource [26] or #CNOT [30], we propose a novel compilation method emphasizing the suppression of photon loss, informed by the practical constraints of current hardware platforms. Our optimization targets are outlined below:

a) *Number of emitter-emitter CNOTs:* Considering gates durations on hardware, emitter-emitter CNOTs are typically more challenging to implement than other operations. As discussed in the quantum dot example in Section II.B, the emitter-emitter CNOT time is more than $10\times$ longer than that of the emitter-photon CNOT, constituting the primary contributor to the overall circuit duration.

b) *Photon loss duration*: Another critical problem in graph state generation is the loss of photon qubits. From the moment a photon is emitted until the entire graph state is fully generated, the photon accumulates loss, reducing the fidelity of the final graph state. Minimizing the existing time of photons can suppress the probability of photon loss. Specifically, given two circuits with the same depth (duration), we prefer the circuit that delays photon emission operations as much as possible.

Compilation Strategy. Based on the time-reversed model, our compilation of each subgraph involves the following steps:

① We search for the graph state reduction sequence, utilizing the reduction operations detailed in Section II.C, considering the maximum number of available emitter ne_{limit} . A heuristic-based depth-first searching is performed to find reduction sequences requiring the minimal number of *emitter disentangle* operations, which corresponds to circuits with minimal emitter-emitter CNOTs. The degree of vertices is used as heuristic, prioritizing the reduction of lower-degree vertices to ensure that more entanglements are created by photon emissions rather than emitter-emitter CNOTs. Additionally, we truncate the operation sequences with a high number of emitter-emitter CNOTs to reduce the searching overhead.

② After obtaining a set of candidate sequences with minimal number of emitter-emitter CNOTs, these sequences are translated into circuits, and the one with the minimal photon loss duration is selected. Specifically, the average photon loss duration ($\bar{T}_{loss}$) is calculated by:

$$\bar{T}_{loss} = \frac{1}{n} \sum_{p=p_1, \dots, p_n} M_{circ_end} - M_{emit}(p)$$

where M_{circ_end} represents the circuit ending time, and $M_{emit}(p)$ represents the emission time of photon p .

Flexible Resource Constraint. We adopt a flexible resource constraint, enabling the generation of optimal circuits under varying numbers of available emitters. For a subgraph g , the minimal number of required emitters, ne_{min} , is calculated using entanglement entropy theory [26]. The subgraph is then compiled with different available emitter limits: $ne_{limit} = ne_{min}, ne_{min} + 1, ne_{min} + 2$, respectively, to generate circuits for each scenario. This approach enhances parallelism within the subgraph when additional emitter qubits are available, facilitating qubit reuse, which will be discussed further in the next subsection.

C. Subgraph Recombination & Circuit Scheduling

Finally, we combine the generation circuits corresponding to each subgraph and schedule these subcircuits. In this step, inter-subgraph entanglements are added to the overall circuit to connect all components. This process is illustrated in Figure 8.

Circuit Scheduling Strategy. Our scheduling scheme adopts the *as-late-as-possible* strategy from Qiskit [34], appending subgraph circuits starting from the latest possible time and working gradually backward. When selecting circuits to schedule, we prioritize subcircuits with more photon qubits and shorter execution times (placing them later), and deprioritize subcircuits with fewer photon qubits and longer execution times (placing them earlier). The priority coefficient for a subgraph circuit is calculated as: $P_c = \frac{n_p^c}{T_c^c}$, where n_p^c is the number of photons in it, and T_c is its duration. This strategy ensures that more photons are emitted as late as possible, minimizing the time lasting for photon loss.

Emitter Qubit Reuse. The timestamp for a subgraph circuit c is determined based on its emitter qubit usage curve. When inserting c into the timeline of overall circuit C , we shift c as late as possible, while ensuring that the total emitter usage in C does not exceed

the maximum emitter limit Ne_{limit} at any moment. This approach is analogous to playing Tetris, where the pieces (subcircuits c) are shifted downward (later in time) as far as possible, while being constrained by the walls (the Ne_{limit}), as shown in Figure 8.

Full Utilization of Emitter Qubits. To maximize the circuit parallelism in Ne_{limit} constraint, we aim to utilize any remaining (unused) emitter qubits at each time slot. This approach works in conjunction with the *flexible resource constraint* policy in Section IV.B. For example, if no additional subgraph circuit c can fit within a given time period, but the total emitter usage does not fully reach Ne_{limit} , we relax the ne_{limit} constraint from ne_{min} to $ne_{min} + 1$ for the circuits already scheduled in that period. This ensures full utilization of emitter qubit resources, ultimately reducing circuit duration.

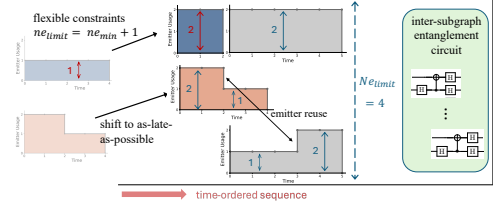


Fig. 8: Our scheduling scheme. The blocks are emitter usage curve for each subgraph, and the width is restricted by total emitters Ne_{limit} .

V. EVALUATION

We evaluate our framework on various benchmarks using several hardware-realistic metrics and compare our results with the state-of-the-art graph state generation framework, GraphiQ [30].

A. Experimental Setup

Benchmarks. The graph states we tested (Figure 9) are selected from popular quantum applications: 1) *Lattice graph* is a two-dimensional (2D) square grid structure, which is basic element for MBQC [8]; 2) *Tree graph* is a connected acyclic graph, which is particularly useful as it constitutes the quantum routers in quantum random access memory (QRAM) [35], while it's also vital to the tree code [36] in quantum error correction; 3) *Random graph* is generated by the Waxman graph structure [37], which covers most of the possible communication topologies for distributed quantum computing and quantum network [38].

Experiment Platform. All compilation and simulation are executed on a CentOS 7 server featuring dual Intel Xeon 8255C CPUs (48 logical cores) and 384GB memory. The Python version is 3.10.0. Our framework is built based on GraphiQ[30] and Qiskit [34].

Hardware Model. We select the silicon quantum dot (QD) emitter [10] as our hardware model for simulation. However, it can be easily adapted to other hardware platform, e.g. SiV color centers [39], NV color centers [14] and Rydberg atoms [15], just by changing the configurations of gate characteristic. In QDs hardware, an emitter-emitter CNOT is realized by electron exchange coupling between two QDs, with coefficient J as the exchange interaction strength. Within a time period $\tau_1 = \pi/2J$, the interaction Hamiltonian yields a $\sqrt{SWAP}$ gate, enabling the entangling between emitters. Then, a emitter-emitter CNOT consists of two $\sqrt{SWAP}$ interleaved by two $R_x \otimes R_x$, while each of them take τ_1 as evolution time. Hence, we



Fig. 9: The types of graph in our benchmarks.

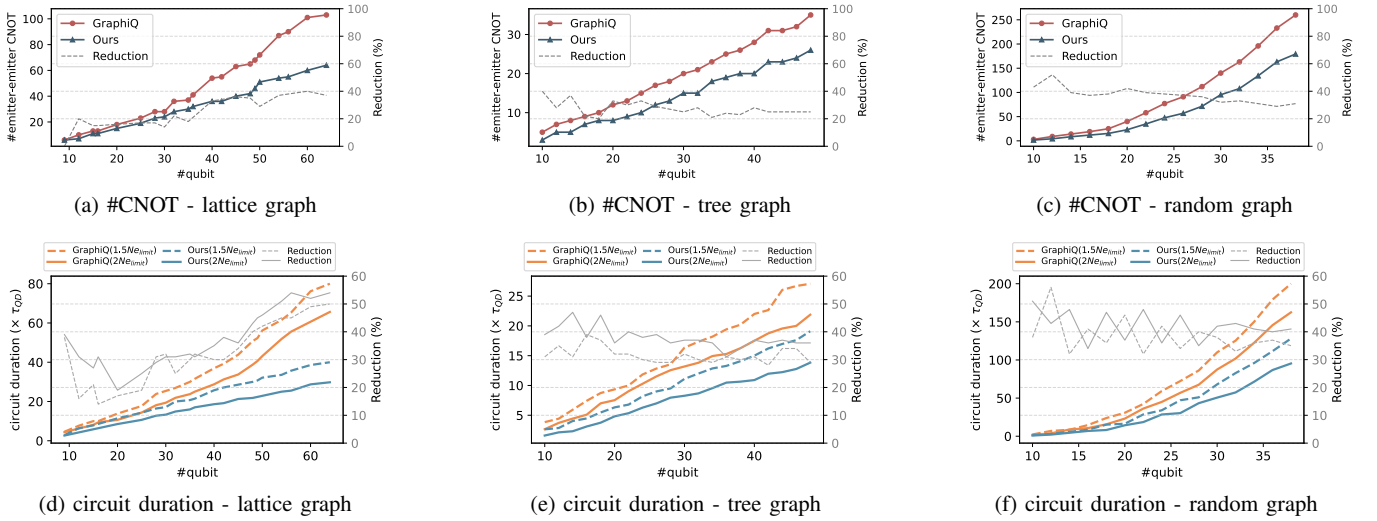


Fig. 10: Evaluation: reduction of emitter-emitter CNOTs and circuit duration.

represent the period of emitter-emitter CNOT as $\tau_{QD} = 4T_1 = 2\pi/J$. The timescale of τ_{QD} depends on J in hardware system, while it is commonly set to $2\pi \times 1\text{GHz}$ and thus $\tau_{QD} = 1\text{ns}$. On the other hand, we set the time period for photon emission as $0.1\tau_{QD}$ based on cavity enhancement [40].

Algorithm Configuration & Baseline. In our framework, we use Gurobi [41] as the MIP solver for graph state partitioning. For the graph partitioning search algorithm in Section IV.A, we set a 20-minute timeout for MIP solver, using the best solution found within this time. In MIP model, we limit the size of subgraph $g_{max} = 7$ and maximum length of LC sequence to $l = 15$. Note that the compilation runtime for each subgraph is relatively low (< 100 seconds), owing to the constrained subgraph size. For baseline GraphIQ, we select its *AlternateTargetSolver* and modify it to include a 30-minute timeout, instead of searching exhaustively.

B. Result

1) *# Emitter-emitter CNOTs:* As the emitter-emitter CNOT being a critical component of graph state generation circuit, we compare its count in the compilation of graph state in varying sizes. The results are shown in Figure 10.(a)-(c) for three types of graphs. Our compilation framework consistently reduces the #CNOT compared to GraphIQ across all graph states, with average reductions of 25%, 28%, 37% (maximum 40%, 39%, 52%) for lattice, tree, and random graphs, respectively. Furthermore, as the size of the graph state increases, the reduction rate grows for lattice graphs and remains steady for the others, demonstrating the scalability of our method.

2) *Circuit Duration:* The time unit we use for calculating duration is τ_{QD} introduced in last subsection, with its realistic value dependent on hardware. We compare circuit durations under two emitter resource settings: $N_{e_{limit}} = 1.5N_{e_{min}}$ and $N_{e_{limit}} = 2N_{e_{min}}$ ($N_{e_{min}}$ is the minimal #emitter required for target graph).

Our results in Figure 10.(d)-(f) show that our compilation framework reduces circuit duration significantly. For $N_{e_{limit}} = 1.5N_{e_{min}}$, the average reduction percentages are 33%, 32%, and 39% (maximum 50%, 39%, and 56%). For $N_{e_{limit}} = 2N_{e_{min}}$, the average reductions are 38%, 38%, and 43% (maximum 54%, 47%, and 51%).

For the same graph state, circuit duration reduction rates are consistently higher than #CNOT reduction rates. Additionally, the duration reduction rate generally increases when the emitter resource setting $N_{e_{limit}}$ is raised from $1.5N_{e_{min}}$ to $2N_{e_{min}}$. This highlights the benefits of circuit parallelism, with its impact becoming more pronounced as more emitter resources are made available.

3) *Photon Loss:* We assess the photon loss rate of final graph state generated by circuit, based on realistic hardware settings. Given the electron spin coherence time $T_2 \sim 1\text{s}$, we set the photon loss rate to 0.5% per time period τ_{QD} . Our simulations ($N_{e_{limit}} = 1.5N_{e_{min}}$) are performed using GraphIQ, and the results are given in Figure 11.(a). Our framework improves the baseline by $\times 1.3$, $\times 1.4$, and $\times 1.9$ on average for three types of graph, respectively.

4) *Local Complementation:* We compare the average number of inter-subgraph edges with and without local complementation optimizing in Figure.11.(b) (by setting LC sequence $l = 15$ or 0), to illustrate the benefits of local complementation. In this experiment, we use random (Waxman) graphs as the benchmark.

VI. CONCLUSION

We propose a novel compilation framework for emitter-photonic graph state generation, leveraging a divide-and-conquer strategy. The framework partitions the graph state into subgraphs, compiles them separately, and then recombines them using circuit scheduling. Tested on various graph states, our framework shows reductions in #CNOT and circuit duration, taking hardware constraints into account.

ACKNOWLEDGMENT

We thanks Liang Jiang for helpful discussion and we acknowledge support from the ARO MURI (W911NF-21-1-0325) and NSF (ERC-1941583). Also, we thanks the reviewers of DAC25 for their invaluable feedback.

REFERENCES

- [1] D. Schlingemann and R. F. Werner, "Quantum error-correcting codes associated with graphs," *Physical Review A*, vol. 65, no. 1, p. 012308, 2001.

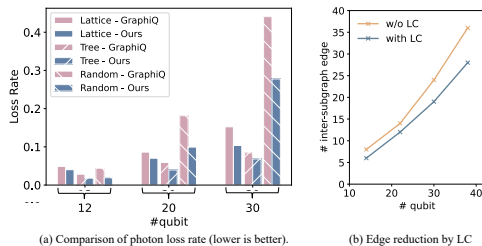


Fig. 11: Evaluation: photon loss suppression and LC optimization.

- [2] R. Raussendorf and H. J. Briegel, "A one-way quantum computer," *Physical review letters*, vol. 86, no. 22, p. 5188, 2001.
- [3] K. Liu, Y. Zhou, H. Luo, L. Xiong, Y. Zhu, E. Casey, J. Cheng, S. Y.-C. Chen, and Z. Liang, "Ecdqc: Efficient compilation for distributed quantum computing with linear layout," *arXiv preprint arXiv:2410.23857*, 2024.
- [4] D. Ferrari and M. Amoretti, "A simulation framework for distributed quantum computing," *arXiv preprint arXiv:2306.11539*, 2023.
- [5] C. D. Donne, M. Iuliano, B. van der Vecht, G. M. Ferreira, H. Jirovská, T. van der Steenhoven, A. Dahlberg, M. Skrzypczyk, D. Fioretto, M. Teller et al., "Design and demonstration of an operating system for executing applications on quantum network nodes," *arXiv preprint arXiv:2407.18306*, 2024.
- [6] D. E. Browne and T. Rudolph, "Resource-efficient linear optical quantum computation," *Physical Review Letters*, vol. 95, no. 1, p. 010501, 2005.
- [7] H. Zhang, J. Ruan, H. Shapourian, R. R. Kompella, and Y. Ding, "Oneperc: A randomness-aware compiler for photonic quantum computing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 738–754. [Online]. Available: <https://doi.org/10.1145/3620666.3651372>
- [8] Y. Li, A. Pawar, Z. Mo, Y. Zhang, J. Yang, and X. Tang, "Minimizing photonic cluster state depth in measurement-based quantum computing," *arXiv preprint arXiv:2312.10865*, 2023.
- [9] Z. Mo, Y. Li, A. Pawar, X. Tang, J. Yang, and Y. Zhang, "Fcm: A fusion-aware wire cutting approach for measurement-based quantum computing," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3657346>
- [10] A. Russo, E. Barnes, and S. E. Economou, "Photonic graph state generation from quantum dots and color centers for quantum communications," *Physical Review B*, vol. 98, no. 8, p. 085303, 2018.
- [11] Y. Arakawa and M. J. Holmes, "Progress in quantum-dot single photon sources for quantum information technologies: A broad spectrum overview," *Applied Physics Reviews*, vol. 7, no. 2, 2020.
- [12] D. Cogan, Z.-E. Su, O. Kenneth, and D. Gershoni, "Deterministic generation of indistinguishable photons in a cluster state," *Nature Photonics*, vol. 17, no. 4, pp. 324–329, 2023.
- [13] K. Nemoto, M. Trupke, S. J. Devitt, A. M. Stephens, B. Scharfenberger, K. Buczak, T. Nöbauer, M. S. Everitt, J. Schmiedmayer, and W. J. Munro, "Photonic architecture for scalable quantum information processing in diamond," *Physical Review X*, vol. 4, no. 3, p. 031022, 2014.
- [14] H. Choi, M. Pant, S. Guha, and D. Englund, "Percolation-based architecture for cluster state creation using photon-mediated entanglement between atomic memories," *npj Quantum Information*, vol. 5, no. 1, p. 104, 2019.
- [15] C.-W. Yang, Y. Yu, J. Li, B. Jing, X.-H. Bao, and J.-W. Pan, "Sequential generation of multiphoton entanglement with a rydberg superatom," *Nature Photonics*, vol. 16, no. 9, pp. 658–661, 2022.
- [16] P. Thomas, L. Ruscio, O. Morin, and G. Rempe, "Efficient generation of entangled multiphoton graph states from a single atom," *Nature*, vol. 608, no. 7924, pp. 677–681, 2022.
- [17] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 1001–1014.
- [18] S. S. Tannu and M. Qureshi, "Ensemble of diverse mappings: Improving reliability of quantum computers by orchestrating dissimilar mistakes," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 253–265.
- [19] S. S. Tannu and M. K. Qureshi, "Not all qubits are created equal: A case for variability-aware policies for nisq-era quantum computers," in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 987–999.
- [20] A. Molavi, A. Xu, M. Diges, L. Pick, S. Tannu, and A. Albarghouthi, "Qubit mapping and routing via maxsat," in *2022 55th IEEE/ACM international symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1078–1091.
- [21] Y. Shi, N. Leung, P. Gokhale, Z. Rossi, D. I. Schuster, H. Hoffmann, and F. T. Chong, "Optimized compilation of aggregated instructions for realistic quantum computers," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1031–1044. [Online]. Available: <https://doi.org/10.1145/3297858.3304018>
- [22] X. Ren, T. Zhang, X. Xu, Y.-C. Zheng, and S. Zhang, "Invited: Leveraging machine learning for quantum compilation optimization," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3663510>
- [23] W.-H. Tseng, Y.-W. Chang, and J.-H. R. Jiang, "Satisfiability modulo theories-based qubit mapping for trapped-ion quantum computing systems," in *Proceedings of the 2024 International Symposium on Physical Design*, 2024, pp. 245–253.
- [24] J. Zhou, Y. Liu, Y. Shi, A. Javadi-Abhari, and G. Li, "Bose-hedral: Compiler optimization for bosonic quantum computing," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 261–276.
- [25] H. Wang, P. Liu, D. B. Tan, Y. Liu, J. Gu, D. Z. Pan, J. Cong, U. A. Acar, and S. Han, "Atomique: A quantum compiler for reconfigurable neutral atom arrays," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 293–309.
- [26] B. Li, S. E. Economou, and E. Barnes, "Photonic resource state generation from a minimal number of quantum emitters," *npj Quantum Information*, vol. 8, no. 1, p. 11, 2022.
- [27] S. Ghanbari, J. Lin, B. MacLellan, L. Robichaud, P. Roztock, and H.-K. Lo, "Optimization of deterministic photonic graph state generation via local operations," *arXiv preprint arXiv:2401.00635*, 2024.
- [28] E. Kaur, A. Patil, and S. Guha, "Resource-efficient and loss-aware photonic graph state preparation using an array of quantum emitters, and application to all-photonic quantum repeaters," *arXiv preprint arXiv:2402.00731*, 2024.
- [29] T. Ji, J. Liu, and Z. Zhang, "Distributing arbitrary quantum cluster states by graph transformation," *arXiv preprint arXiv:2404.05537*, 2024.
- [30] J. Lin, B. MacLellan, S. Ghanbari, J. Belleville, K. Tran, L. Robichaud, R. G. Melko, H.-K. Lo, and P. Roztock, "Graphiq: Quantum circuit design for photonic graph states," *arXiv preprint arXiv:2402.09285*, 2024.
- [31] M. Gimeno-Segovia, T. Rudolph, and S. E. Economou, "Deterministic generation of large-scale entangled photonic cluster state from interacting solid state emitters," *Physical review letters*, vol. 123, no. 7, p. 070501, 2019.
- [32] A. Dahlberg, J. Helsen, and S. Wehner, "Counting single-qubit clifford equivalent graph states is # p-complete," *Journal of Mathematical Physics*, vol. 61, no. 2, 2020.
- [33] X. Ren, M. Zhang, and A. Barbalace, "A hardware-aware gate cutting framework for practical quantum circuit knitting," *arXiv preprint arXiv:2409.03870*, 2024.
- [34] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," 2024.
- [35] S. Xu, C. T. Hann, B. Foxman, S. M. Girvin, and Y. Ding, "Systems architecture for quantum random access memory," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 526–538.
- [36] K. Azuma, K. Tamaki, and H.-K. Lo, "All-photonic quantum repeaters," *Nature communications*, vol. 6, no. 1, pp. 1–7, 2015.
- [37] B. M. Waxman, "Routing of multipoint connections," *IEEE journal on selected areas in communications*, vol. 6, no. 9, pp. 1617–1622, 1988.
- [38] Y. Huang, X. Ren, B. Li, Y. Wong, and L. Jiang, "Space-time peer-to-peer distribution of multi-party entanglement for any quantum network," *arXiv preprint arXiv:2412.14757*, 2024.
- [39] P.-J. Stas, Y. Q. Huan, B. Machielse, E. N. Knall, A. Suleymanzade, B. Pingault, M. Sutula, S. W. Ding, C. M. Knaut, D. R. Assumpcao et al., "Robust multi-qubit quantum network node with integrated error detection," *Science*, vol. 378, no. 6619, pp. 557–560, 2022.
- [40] Y. A. Kelaita, K. A. Fischer, T. M. Babinec, K. G. Lagoudakis, T. Sarmiento, A. Rundquist, A. Majumdar, and J. Vučković, "Hybrid metal-dielectric nanocavity for enhanced light-matter interactions," *Optical Materials Express*, vol. 7, no. 1, pp. 231–239, 2016.
- [41] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024. [Online]. Available: <https://www.gurobi.com>